

VEC Research Laboratory

Module 1

Building a Vehicular Edge Computing Simulator from Scratch

Dr. Habte Lejebo

Department of Computer Science / Software Engineering

[Institution]

*Research Areas: Vehicular Edge Computing · Fog Computing · Deep Reinforcement
Learning*

A hands-on research laboratory for Master's students and early PhD researchers entering the field of Vehicular Edge Computing and AI-driven task scheduling.

Contents

Preface	3
1 Introduction to Vehicular Edge Computing	4
1.1 The Problem — Why Vehicles Need Computing	4
1.2 The Research Problem	5
1.3 What You Will Build in Module 1	5
2 Object-Oriented Modeling of VEC Components	6
2.1 Why Object-Oriented Design?	6
2.2 The Task Entity	6
2.3 The Vehicle Entity	8
2.4 The EdgeServer Entity	9
3 Task Generation Model	11
3.1 Stochastic Task Arrivals	11
3.2 Task Size Distribution	11
3.3 Reproducibility — Random Seeds	12
4 Computation Model	13
4.1 The Discrete Time Step	13
4.2 Processing Delay	13
4.3 Local vs Edge Execution	14
5 Queueing System	15
5.1 Why Queues Are Necessary	15
5.2 FIFO Queue Implementation	15
5.3 The Complete Simulation Loop	15
6 Metrics and Evaluation	17
6.1 Research Metrics	17
6.2 Task Accounting	17
7 Laboratory Exercises	19
7.1 Understanding Questions	19
7.2 Implementation Exercises	20
7.3 Research Challenges	21

A	Reference Implementation	22
A.1	File Structure	22
A.2	GitHub Repository Structure for Lab Manual	22
A.3	Module 2 Preview	23

Preface

This laboratory manual documents a complete research journey — from a blank Python file to a working VEC simulator equipped with a Deep Reinforcement Learning scheduling agent. It is designed for Master’s students and early PhD researchers who want to enter the field of Vehicular Edge Computing (VEC) and Mobile Edge Computing (MEC).

Most research papers present only the final algorithm and results. They do not show how the simulator was designed, what design mistakes were made, how they were corrected, and why specific architectural decisions were taken. This manual fills that gap.

Each chapter has two layers:

- **Learning Layer** — explains what problem is being solved, why each component exists, and how the code represents the real system.
- **Research Note** — maps the implementation to IEEE-style mathematical formulations and points to possible research extensions.

RESEARCH DIARY — Why not use SUMO or ns-3?

A common mistake new researchers make is starting with a complex simulator before understanding the fundamental models. This module first builds a minimal simulator from scratch so that every component is understood before adding complexity. Once you understand the core dynamics, integrating tools like SUMO for realistic mobility becomes straightforward. The best research tools are the ones you understand deeply, not the most sophisticated ones available.

Chapter 1

Introduction to Vehicular Edge Computing

1.1 The Problem — Why Vehicles Need Computing

Modern vehicles are no longer simple transportation machines. A self-driving car generates between 1 and 20 terabytes of sensor data per day from cameras, LiDAR, radar, and GPS. Each piece of data requires real-time processing — object detection, path planning, hazard recognition — with latency requirements measured in milliseconds.

The question is: *where does this computation happen?*

Table 1.1: Comparison of computing locations for vehicular tasks

Location	Latency	Capacity	Limitation
On-vehicle (local)	< 1 ms	Low	Limited CPU, drains battery
Edge server (RSU)	1–10 ms	Medium	Coverage limited by distance
Cloud server	50–200 ms	High	Too slow for safety-critical tasks

Edge computing places computational resources close to the vehicle — at roadside units (RSUs), base stations, or smart traffic infrastructure. This reduces latency while offloading computational burden from the vehicle CPU.

RESEARCH NOTE — Formal VEC System Definition

A VEC system is formally defined by three sets:

$$\mathcal{V} = \{v_1, v_2, \dots, v_N\} \quad \text{— the set of } N \text{ mobile vehicles} \quad (1.1)$$

$$\mathcal{R} = \{r_1, r_2, \dots, r_M\} \quad \text{— the set of } M \text{ edge servers (RSUs)} \quad (1.2)$$

$$\mathcal{T} = \{t_1, t_2, \dots, t_K\} \quad \text{— the set of } K \text{ computational tasks} \quad (1.3)$$

The scheduling problem asks: for each task $t_i \in \mathcal{T}$, decide whether to execute it locally on the vehicle or offload it to an RSU $r_j \in \mathcal{R}$, such that a performance objective (e.g., task completion rate, average latency) is maximised.

1.2 The Research Problem

Vehicular Edge Computing introduces three fundamental research challenges that simple heuristics cannot solve:

1. **Mobility** — vehicles move continuously, changing their distance from RSUs and therefore their wireless channel quality and available servers.
2. **Heterogeneous resources** — vehicle CPUs and edge servers have different capacities, making the optimal offloading decision non-trivial.
3. **Dynamic workload** — task arrival is stochastic; the scheduler must adapt to varying loads in real time.

These three properties make VEC scheduling an ideal application for Deep Reinforcement Learning — an agent that learns an adaptive policy through repeated interaction with the environment.

RESEARCH DIARY — The research question this manual answers

“Can a DQN-based scheduling agent learn to balance local and edge resources adaptively, outperforming fixed-policy baselines in a realistic VEC environment with mobility, wireless channel degradation, and server handover?”

By the end of Module 2, you will have experimental evidence to answer this question, including a 17.7 percentage point improvement over the always-offload baseline.

1.3 What You Will Build in Module 1

By the end of this module, you will have built a working VEC simulator with:

- A vehicle that moves in 2D space and generates computational tasks stochastically.
- An edge server that processes tasks with realistic computation delay.
- A task queue that never loses tasks — accepted tasks wait until the server is free.
- A simulation loop that advances time step by step, observable at each step.
- Basic metrics: completion rate, average task latency, queue length.

IMPORTANT — Development Philosophy

Every component in this manual is built incrementally. We start with the smallest possible working version and add complexity only when a real requirement appears. The principle is: **Build** → **Test** → **Understand** → **Refactor** → **Expand**. This mirrors how research simulators are actually developed.

Chapter 2

Object-Oriented Modeling of VEC Components

2.1 Why Object-Oriented Design?

A VEC simulator is a world of interacting entities — vehicles, servers, tasks, schedulers. Object-Oriented Programming (OOP) allows us to model each entity as an independent object that knows its own state and can perform its own actions.

This matters for research because:

- Each component can be modified independently without breaking others — a property called *Separation of Concerns*.
- New components can be added incrementally without rewriting existing code.
- The code structure mirrors the mathematical model, making it easier to translate between implementation and paper.

RESEARCH DIARY — The most important design decision

When we added the wireless channel model in Sprint 13 of the development, only the simulation loop changed. `Task`, `Vehicle`, and `EdgeServer` remained completely untouched. That is the practical benefit of Separation of Concerns — future work costs less time.

2.2 The Task Entity

A task represents a unit of computation that a vehicle needs to execute. Before writing any code, define what a task *knows* about itself:

- What computation does it require? → `cpu_cycles`
- Who generated it? → `owner_id`
- Is it time-sensitive? → `deadline`
- What is its current lifecycle state? → `status`
- How much of its work remains? → `remaining_cycles`

RESEARCH NOTE — Mathematical Task Model

Each task t_i is characterised by the tuple:

$$t_i = (\phi_i, d_i, D_i) \quad (2.1)$$

where ϕ_i is the CPU cycle requirement, d_i is the data size in bits, and D_i is the deadline. The computation delay when processed by a resource with capacity F (cycles per time step) is:

$$T_i^{\text{comp}} = \left\lceil \frac{\phi_i}{F} \right\rceil \text{ steps} \quad (2.2)$$

```

1  class Task:
2      def __init__(self, task_id, owner_id, cpu_cycles, deadline)
3          :
4              self.task_id          = task_id
5              self.owner_id         = owner_id          # which
6                  vehicle created this
7              self.cpu_cycles       = cpu_cycles        # total work
8                  required (phi_i)
9              self.remaining_cycles = cpu_cycles        # counts
10                  down during processing
11              self.deadline         = deadline          # maximum
12                  allowed latency (D_i)
13              self.data_size        = cpu_cycles * 1000 # bytes
14                  (d_i)
15              self.status           = "pending"         # lifecycle
16                  state
17              self.created_at        = None              # set when
18                  generated
19              self.completed_at      = None              # set when
20                  finished
21              self.latency           = None              #
22                  completed_at - created_at
23              self.transmission_remaining = 0
24              self.backhaul_remaining = 0
25
26      def __repr__(self):
27          return (f"Task {self.task_id} | owner: Vehicle {self.
28                  owner_id} | "
29                  f"cpu: {self.cpu_cycles} | status: {self.status
30                  }")

```

Listing 2.1: task.py — The Task class

The lifecycle of a task follows the state machine:

pending → transmitting → queued → processing → completed

2.3 The Vehicle Entity

A vehicle in a VEC simulator is not just a physical object that moves — it is a computational node. It generates tasks, has a local CPU for processing, and maintains a communication link to nearby edge servers.

RESEARCH NOTE — Mobility Model

The 2D position update follows the constant-velocity model:

$$x(t+1) = x(t) + v \cdot \cos(\theta), \quad y(t+1) = y(t) + v \cdot \sin(\theta) \quad (2.3)$$

where v is speed and θ is direction in radians. This is a simplified model; real VEC research often uses the Random Waypoint Model or SUMO-generated traces.

```

1  import math, random
2  from task import Task
3
4  class Vehicle:
5      def __init__(self, vehicle_id, x, y, speed, direction):
6          self.vehicle_id      = vehicle_id
7          self.x                = x
8          self.y                = y
9          self.speed            = speed
10         self.direction         = direction      # degrees
11         self.local_capacity    = 150            # CPU cycles/
12         self.local_task        = None
13         self.local_queue       = []
14         self.local_completed   = []            # per-step
15         self.local_tasks_processed = 0
16         self.task_counter      = 0
17
18     def move(self):
19         """Update position based on speed and direction."""
20         self.x += self.speed * math.cos(math.radians(self.
21             direction))
22         self.y += self.speed * math.sin(math.radians(self.
23             direction))
24
25     def generate_tasks(self, current_step):
26         """Generate 0-3 tasks this time step (stochastic
27             arrival)."""
28         tasks = []
29         for _ in range(random.randint(0, 3)):
30             task = Task(
31                 task_id      = f"{self.vehicle_id}-{self.
32                     task_counter}",
33                 owner_id     = self.vehicle_id,
34                 cpu_cycles= random.randint(200, 600),

```

```

31         deadline = 20
32     )
33     task.created_at = current_step
34     self.task_counter += 1
35     tasks.append(task)
36     return tasks

```

Listing 2.2: vehicle.py — The Vehicle class (core methods)

2.4 The EdgeServer Entity

An edge server (RSU) is a fixed infrastructure node that provides computational resources to nearby vehicles. It has a known position, a coverage radius, and a processing capacity.

RESEARCH NOTE — Shannon Capacity and Coverage

The wireless data rate between a vehicle at distance d from an RSU follows:

$$R = B \log_2(1 + \text{SNR}), \quad \text{SNR} = \frac{P}{N_0 \cdot d^2} \quad (2.4)$$

where B is bandwidth, P is transmission power, and N_0 is the noise factor. As distance increases, SNR decreases non-linearly (d^2 in denominator), causing the rate to drop sharply near the coverage boundary.

```

1  import math
2
3  class EdgeServer:
4      def __init__(self, server_id, capacity, x=0, y=0,
5                  coverage_radius=10, backhaul_bandwidth=10
6                  _000_000):
7          self.server_id = server_id
8          self.capacity = capacity # CPU cycles/
9                                step (F)
10         self.x = x
11         self.y = y
12         self.coverage_radius = coverage_radius
13         self.backhaul_bandwidth = backhaul_bandwidth # bytes/
14                                second
15         self.current_task = None
16         self.task_queue = []
17         self.transmission_queue = []
18         self.backhaul_queue = []
19         self.completed_tasks = [] # per-step event buffer
20         self.tasks_processed = 0
21
22     def in_range(self, vehicle):
23         """Return True if vehicle is within wireless coverage."""
24         ""
25         d = math.sqrt((self.x - vehicle.x)**2 + (self.y -
26             vehicle.y)**2)

```

```
22         return d <= self.coverage_radius
23
24     def reset(self):
25         """Reset all state between training episodes."""
26         self.current_task = None
27         self.task_queue = []
28         self.transmission_queue = []
29         self.backhaul_queue = []
30         self.completed_tasks = []
31         self.tasks_processed = 0
```

Listing 2.3: edge_server.py — Key methods of EdgeServer

Chapter 3

Task Generation Model

3.1 Stochastic Task Arrivals

In a real vehicular system, tasks do not arrive at a fixed rate. A vehicle may generate three tasks in one second and zero in the next. We model task arrivals as a discrete uniform random process: at each time step, the vehicle generates between 0 and 3 tasks with equal probability.

```
1 num_tasks = random.randint(0, 3)
```

Listing 3.1: Stochastic task generation

This gives an average arrival rate of:

$$\lambda = \frac{0 + 1 + 2 + 3}{4} = 1.5 \text{ tasks per step} \quad (3.1)$$

RESEARCH NOTE — Arrival Rate and System Stability

The system utilisation factor $\rho = \lambda/\mu$ determines queue stability:

$$\rho < 1 \Rightarrow \text{stable (queue drains over time)} \quad \rho \geq 1 \Rightarrow \text{unstable (queue grows without bound)} \quad (3.2)$$

In our simulator, $\lambda = 1.5$ tasks/step and combined service rate $\mu \approx 0.83$ tasks/step, giving $\rho \approx 1.81$. This represents a moderately overloaded system where intelligent scheduling provides a significant advantage over fixed policies. An underloaded system ($\rho \ll 1$) requires no intelligence to schedule optimally — it is not a research-worthy problem.

3.2 Task Size Distribution

Task size determines processing time. We sample uniformly from a range representing realistic vehicular computing workloads:

```
1 cpu_cycles = random.randint(200, 600)
```

Listing 3.2: Task size sampling

With server capacity 300 cycles/step, a task takes $\lceil 400/300 \rceil = 2$ steps on average. Locally (capacity 150), it takes $\lceil 400/150 \rceil = 3$ steps.

RESEARCH DIARY — Why not use `randint(200, 1000)`?

Our first implementation used a range of 200–1000. This made the average task size 600 cycles, requiring 4 steps locally. With arrival rate 1.5 and local service rate 0.25 tasks/step, the local queue grew unboundedly — the system was pathologically overloaded and the agent learned nothing meaningful. Reducing to 200–600 fixed stability while keeping the scheduling problem non-trivial. This is called *workload shaping* in the queueing theory literature, and it is the correct research move: adjust job size distribution rather than system architecture.

3.3 Reproducibility — Random Seeds

A key requirement in research simulation is reproducibility.

```
1 import random, numpy as np
2
3 random.seed(42)           # Python random module
4 np.random.seed(42)        # NumPy random module
5
6 # Use a different seed for evaluation (tests generalisation)
7 # Training: seed=42      Evaluation: seed=123
```

Listing 3.3: Setting random seeds for reproducibility

Chapter 4

Computation Model

4.1 The Discrete Time Step

Unlike real-time systems, a discrete-event simulator advances time in fixed steps. Each step represents 0.1 seconds of simulated time. Within each step:

1. The vehicle moves to its new position.
2. The vehicle's local CPU processes its current task.
3. Each edge server processes its current task.
4. New tasks are generated and scheduling decisions are made.

4.2 Processing Delay

A task requires multiple time steps to process:

```
1 def tick(self, current_step):
2     # Pull from queue if server is idle
3     if self.current_task is None and self.task_queue:
4         next_task = self.task_queue.pop(0)
5         next_task.status = "processing"
6         self.current_task = next_task
7
8     if self.current_task is None:
9         return
10
11     # Reduce remaining work
12     self.current_task.remaining_cycles -= self.capacity
13
14     # Check if task is complete
15     if self.current_task.remaining_cycles <= 0:
16         self.current_task.status = "completed"
17         self.current_task.completed_at = current_step
18         self.current_task.latency = (current_step
19                                     - self.current_task.
20                                     created_at)
21
22     self.tasks_processed += 1
```

```

21     self.completed_tasks.append(self.current_task)
22     self.current_task = None

```

Listing 4.1: EdgeServer tick() — one step of computation

RESEARCH NOTE — End-to-End Task Latency

The total latency of an offloaded task has three components:

$$T_i^{\text{total}} = T_i^{\text{trans}} + T_i^{\text{queue}} + T_i^{\text{comp}} \quad (4.1)$$

The formula `latency = completed_at - created_at` automatically captures all three, because the task is created before transmission, waits in queue, and only completes after processing. This end-to-end latency is the most meaningful metric for VEC research — it reflects the full user-perceived delay.

4.3 Local vs Edge Execution

A vehicle has two execution options for each task:

- **Local execution** — processed on the vehicle's own CPU (capacity 150 cycles/step). No transmission delay but higher total latency due to lower capacity.
- **Edge offloading** — transmitted wirelessly to the RSU and processed there (capacity 300 cycles/step). Transmission delay added, but faster computation.

The scheduling decision — local or offload — is the core optimisation problem.

Chapter 5

Queueing System

5.1 Why Queues Are Necessary

In our first simulator version, when the edge server was busy, new tasks were simply discarded. This is catastrophically unrealistic — a real system cannot afford to lose tasks silently. A task queue is the waiting room where tasks sit until the server is free.

5.2 FIFO Queue Implementation

```
1 def accept_task(self, task):
2     """Accept a task for processing or queueing."""
3     if self.current_task is None:
4         self.current_task = task
5         task.status = "processing"
6         return True          # accepted directly
7     else:
8         self.task_queue.append(task)    # join the waiting room
9         task.status = "queued"
10        return False         # placed in queue
```

Listing 5.1: EdgeServer accept_task() — FIFO queuing

RESEARCH NOTE — Why DRL Is Needed

A classical M/M/1 queue model assumes a single server, Poisson arrivals, and exponential service times — none of which hold in VEC. Our system has:

- Two heterogeneous processors (vehicle CPU + edge RSU)
- Time-varying channel quality (depends on distance d)
- Non-stationary load (vehicle moves in and out of coverage)

A fixed-policy scheduler cannot adapt to these dynamics. A DRL agent learns an adaptive policy that jointly considers queue states, channel quality, and resource availability at each decision step.

5.3 The Complete Simulation Loop


```

1  import csv, random
2  from vehicle import Vehicle
3  from edge_server import EdgeServer
4
5  random.seed(42)
6
7  vehicle = Vehicle(vehicle_id=1, x=0, y=0, speed=2, direction=0)
8  server = EdgeServer(server_id=1, capacity=300, x=10, y=0,
9                      coverage_radius=10)
10 step_metrics = []
11
12 for step in range(35):
13
14     # 1. Vehicle moves and processes local tasks
15     vehicle.move()
16     vehicle.tick(step)
17
18     # 2. Edge server processes queued tasks
19     server.tick(step)
20
21     # 3. Check wireless coverage
22     if server.in_range(vehicle):
23
24         # 4. Generate and schedule tasks
25         for task in vehicle.generate_tasks(step):
26             server.accept_task(task) # simplified: always
27                                     offload
28
29     step_metrics.append({
30         "step" : step,
31         "vehicle_x" : round(vehicle.x, 1),
32         "queue_length" : len(server.task_queue),
33         "tasks_processed" : server.tasks_processed,
34         "in_range" : server.in_range(vehicle)
35     })
36
37     # Save metrics
38     with open("step_metrics.csv", "w", newline="") as f:
39         writer = csv.DictWriter(f, fieldnames=step_metrics[0].keys
40                                ())
41         writer.writeheader()
42         writer.writerows(step_metrics)
43
44 print(f"Tasks processed: {server.tasks_processed}")

```

Listing 5.2: simulation.py — Minimum viable simulation loop

Chapter 6

Metrics and Evaluation

6.1 Research Metrics

Research simulators must produce measurable outcomes. Three primary metrics are essential for Module 1:

Table 6.1: Core evaluation metrics for Module 1

Metric	Research Significance	Formula
Task completion rate	Primary performance indicator — what fraction of generated tasks actually finish processing	completed/generated
Average task latency	Quality of service indicator — how long each task takes from creation to completion	$\bar{T}^{\text{total}}$
Task drop rate	System reliability indicator — tasks lost due to coverage gaps or queue overflow	dropped/generated

6.2 Task Accounting

A correctly implemented simulator satisfies the accounting identity:

$$\text{generated} = \text{completed} + \text{dropped} + \text{unfinished at episode end} \quad (6.1)$$

```
1 all_completed = (  
2     sum(s.tasks_processed for s in servers.values()) +  
3     vehicle.local_tasks_processed  
4 )  
5  
6 completion_rate = all_completed / total_generated  
7 drop_rate       = total_dropped / total_generated  
8 unfinished_rate = 1.0 - completion_rate - drop_rate  
9  
10 print(f"Completion rate : {completion_rate:.1%}")  
11 print(f"Drop rate      : {drop_rate:.1%}")
```

```
12 print(f"Unfinished      : {unfinished_rate:.1%}")
```

Listing 6.1: Metrics collection at episode end

RESEARCH DIARY — The simulation boundary effect

In a 35-step episode, many tasks are still in queues when the simulation ends. These “unfinished” tasks are not lost — they were accepted and partially processed. They simply did not finish within the evaluation window. This is called the *simulation boundary effect* and must be acknowledged in any research paper reporting completion rate metrics. One mitigation is to extend the episode length; another is to run the simulation until all queues drain completely.

Chapter 7

Laboratory Exercises

7.1 Understanding Questions

Exercise 1: Cloud vs Edge Computing

Why is cloud computing insufficient for safety-critical vehicular applications? Name two specific latency requirements that cloud computing cannot meet and explain why an edge server addresses each one.

Exercise 2: Separation of Concerns

Explain the Separation of Concerns principle in your own words. Give one concrete example from the VEC simulator showing how it was applied, and describe what would happen if it were violated.

Exercise 3: Processing Delay Calculation

A task has `cpu_cycles` = 450. The edge server has `capacity` = 300. The vehicle's local CPU has `local_capacity` = 150.

- How many simulation steps does edge processing take? Show your calculation.
- How many simulation steps does local processing take?
- Which is faster? Does faster always mean better? Explain.

Exercise 4: Queue Stability

What is the stability condition for a queueing system? In our simulator with $\lambda = 1.5$ tasks/step and $\mu = 0.83$ tasks/step, is the system stable? What happens to queue length over time, and why does this motivate the use of DRL?

Exercise 5: End-to-End Latency

Explain why `latency = completed_at - created_at` automatically includes transmission delay, queueing delay, and computation delay. Trace through a specific example: a task created at step 3 that spends 2 steps in the transmission queue, 1 step in the task queue, and 2 steps being processed.

7.2 Implementation Exercises

Exercise 6: Implement the Task class

Complete the following skeleton. Do not look at the reference implementation until you have attempted your own version.

```
1 class Task:
2     def __init__(self, task_id, owner_id, cpu_cycles,
3         deadline):
4         # TODO: Store task_id
5         # TODO: Store owner_id
6         # TODO: Store cpu_cycles
7         # TODO: Initialize remaining_cycles equal to
8             cpu_cycles
9         # TODO: Store deadline
10        # TODO: Initialize status as "pending"
11        # TODO: Initialize created_at, completed_at, latency
12            as None
13        pass
14
15    def __repr__(self):
16        # TODO: Return a descriptive string showing task_id,
17        #         owner_id, cpu_cycles, and status
18        pass
```

Exercise 7: Implement Vehicle.move()

Complete the method that updates vehicle position using speed and direction.

```
1 def move(self):
2     # TODO: Update self.x using speed, cos, and direction
3     # TODO: Update self.y using speed, sin, and direction
4     # HINT: Use math.cos(math.radians(self.direction))
5     pass
```

Verify your implementation by creating a vehicle at (0,0) with speed=2 and direction=0, calling move() three times, and checking that x increases by 2 each time.

Exercise 8: Implement EdgeServer.in_range()

Complete the coverage check using the Euclidean distance formula.

```
1 def in_range(self, vehicle):
2     # TODO: Compute Euclidean distance between server and
3         vehicle
4     # TODO: Return True if distance <= self.coverage_radius
5     pass
```

Exercise 9: Implement the task queue

Complete `accept_task()` to implement FIFO queuing.

```

1  def accept_task(self, task):
2      # TODO: If server is idle (current_task is None):
3      #         set task as current_task
4      #         set task.status = "processing"
5      #         return True
6      # TODO: Otherwise:
7      #         append task to task_queue
8      #         set task.status = "queued"
9      #         return False
10     pass

```

Exercise 10: Build the simulation loop

Using the classes you implemented, build a complete simulation loop that:

- Runs for 35 time steps
- Moves the vehicle each step
- Ticks the server each step
- Generates and offloads tasks when in range
- Prints a summary at the end: tasks generated, processed, and completion rate

7.3 Research Challenges

Exercise 11: Deadline-Aware Task Dropping

Currently, tasks wait in the queue indefinitely. In real VEC systems, a task that cannot complete before its deadline should be dropped. Modify `tick()` to check whether a queued task's deadline will be missed and remove it from the queue if so. Report: how does this change completion rate and average latency?

Exercise 12: Multiple Vehicle Types

Extend the `Vehicle` class to support heterogeneous types: passenger car (speed=2, local_capacity=150), emergency vehicle (speed=4, local_capacity=300), and autonomous bus (speed=1, local_capacity=500). How does the scheduling problem change when vehicles have heterogeneous capabilities?

Exercise 13: Queue Priority Policy

Replace FIFO with Shortest Job First (SJF) — the queue always processes the task with the fewest remaining_cycles next. Implement this and compare completion rate and average latency against FIFO over 100 simulation episodes. Under what conditions does SJF outperform FIFO?

Appendix A

Reference Implementation

A.1 File Structure

```
1 vec_simulator/  
2 |-- task.py           # Task class  
3 |-- vehicle.py        # Vehicle class  
4 |-- edge_server.py    # EdgeServer class  
5 |-- channel.py        # WirelessChannel (Module 2)  
6 |-- agent.py          # DQNAgent (Module 2)  
7 |-- simulation.py     # Main simulation loop  
8 |-- baseline_comparison.py # Policy evaluation  
9 |-- plot.py           # Learning curve visualisation  
10 |-- results/  
11 |   |-- episode_metrics.csv  
12 |   '-- step_metrics.csv  
13 '-- README.md
```

Listing A.1: Recommended project structure

A.2 GitHub Repository Structure for Lab Manual

```
1 VEC-DRL-Lab/  
2 |-- README.md  
3 |-- labs/  
4 |   |-- Lab01_OOP_Modeling/  
5 |     |-- skeleton/      # student starting point (TODO  
6 |       '-- solution/    # reference implementation  
7 |   |-- Lab02_Task_Processing/  
8 |   |-- Lab03_Task_Queue/  
9 |   |-- Lab04_Mobility/  
10 |   '-- Lab05_Multi_RSU/  
11 |-- src/  
12 |   |-- task.py  
13 |   |-- vehicle.py  
14 |   |-- edge_server.py  
15 |   |-- channel.py
```

```
16 | |-- agent.py
17 | '-- simulation.py
18 |-- experiments/
19 | '-- baseline_comparison.py
20 '-- docs/
21 '-- VEC_Lab_Manual_Module1.pdf
```

Listing A.2: GitHub repository layout

A.3 Module 2 Preview

Module 2 extends this simulator with:

- **Wireless channel model** — Shannon capacity, transmission delay, distance-based SNR degradation.
- **Multi-server deployment** — three RSUs with coverage zones, handover, and back-haul.
- **Deep Reinforcement Learning scheduler** — DQN agent that learns to balance local and edge resources.
- **Reward engineering** — ablation study across three reward designs demonstrating the importance of outcome-based rewards.
- **Baseline comparison** — DQN vs always-offload vs always-local across 100 evaluation episodes.

The DQN agent in Module 2 achieves a **59.0%** task completion rate, outperforming the always-offload baseline by **17.7 percentage points** — a result directly enabled by the simulator architecture built in Module 1.